ECOLE NORMALE SUPERIEURE PARIS-SACLAY

Universite Paris-Saclay

SATIE

Laboratoire des systemes et applications des technologies de l'information et de l'energie

Document Recapitulatif

Pipeline de Reconstruction par Diffusion avec Correction EMG-VBA une dimension

Equations, Pseudo-code et Architecture du Code

Gatien Seguy

Laboratoire : SATIE – ENS Paris-Saclay

Superviseur : Thomas Rodet (SATIE – CNRS)

DESCRIPTION

Ce document recapitule l'ensemble des equations et algorithmes implementes dans le code de reconstruction d'images par modeles de diffusion avec correction EMG-VBA. Pour chaque equation, le numero correspondant dans le rapport de stage est indique en bleu. Le pseudo-code algorithmique explicite le lien entre les formules mathematiques et les fichiers Python.

Stage de recherche – Master 2 FESup – Avril 2026

Table des matières

I Vue d'ensemble de la pipeline **3**

II Phase 1 : Entrainement du reseau ε_θ **4**
- **II.1 Schedule DDPM** **4**
- **II.2 Processus forward** **4**
- **II.3 Formule de Tweedie** **5**
- **II.4 Objectif d'entrainement** **5**

III Phase 2 : Degradation de l'image **6**
- **III.1 Modele d'observation** **6**
- **III.2 Construction de l'operateur A** **6**
- **III.3 Precalculs pour EMG-VBA** **6**

IV Phase 3 : Inference conditionnelle (boucle reverse) **7**
- **IV.1 Architecture globale** **7**
- **IV.2 Score conditionnel et μ_{post}** **7**
- **IV.3 Modele hierarchique bayesien** **7**
- **IV.4 Approximation variationnelle** **8**
- **IV.5 Log-vraisemblance jointe** **8**

V Phase 3a : Algorithme EMG-VBA (detail) **8**
- **V.1 Etape 1 : Mise a jour de $\langle \tau_b \rangle$ et $\langle \tau_r \rangle$** **9**
- **V.2 Etape 2 : Distribution de reference $q_i^r(x_{0,i})$** **9**
- **V.3 Etape 3 : Pas sous-optimal** **10**
- **V.4 Etape 4 : Mise a jour de $q(x_0)$** **12**
- **V.5 Energie libre negative (diagnostic)** **13**
- **V.6 Synthese : boucle EMG-VBA interne** **13**
- **V.7 Initialisation de EMG-VBA** **13**

VI Phase 3b : Pas reverse DDPM conditionnel **14**

VII Phase 3c : Boucle reverse complete **15**

VIII Phase 4 : Orchestration **16**

IX Recapitulatif des equations **16**

X Hyperparametres **17**

I Vue d'ensemble de la pipeline

La pipeline complete se decompose en quatre grandes phases :

1. **Entrainement** (`Training.py`) : apprentissage du reseau ε_θ sur des images propres.
2. **Degradation** (`degrade.py`) : construction de l'observation $y = Ax_0$ (flou gaussien).
3. **Inference conditionnelle** (`Reverse.py` + `emg_vba_1_dim.py`) : boucle reverse $t = T, \ldots, 1$ avec correction EMG-VBA a chaque pas.
4. **Orchestration** (`main.py`) : chaine complete degradation → reconstruction → affichage.

Remarque— Convention de normalisation

Toutes les images sont stockees dans $[-1, 1]$ dans le code :

$$x_{[-1,1]} = 2\,x_{[0,1]} - 1, \qquad x_{[0,1]} = \frac{x_{[-1,1]} + 1}{2}.$$

Le reseau ε_θ est entraine sur des images en $[-1, 1]$. La sortie finale est reconvertie en $[0, 1]$ par `clamp(-1,1)` puis $(x + 1)/2$.

Fichiers et dependances :

Fichier	Role	Depend de
`Schedules.py`	Definition de $\beta_t, \alpha_t, \bar{\alpha}_t, \sigma_t$	—
`Forward.py`	Processus forward + Tweedie	`Schedules`
`Training.py`	Boucle d'entrainement (loss MSE + EMA)	`Forward`
`model.py`	Architecture UNet	—
`degrade.py`	Flou gaussien, construction de A	—
`operators.py`	Classe `GaussianBlurOperator` (FFT)	—
`emg_vba_1_dim.py`	Algorithme EMG-VBA complet	—
`Reverse.py`	Reverse DDPM + correction EMG-VBA	`Forward`, `emg_vba_1_dim`
`main.py`	Orchestration	tous

II Phase 1 : Entrainement du reseau ε_θ

II.1 Schedule DDPM

Resultat II.1.1— Definitions [Rapport eq. III.3.1]

$$\beta_t = \text{linspace}(\beta_{\min}, \beta_{\max}, T), \quad t = 0, \dots, T-1 \tag{1}$$
$$\alpha_t = 1 - \beta_t \tag{2}$$
$$\bar{\alpha}_t = \prod_{s=0}^{t} \alpha_s \tag{3}$$
$$\sigma_t = \sqrt{1 - \bar{\alpha}_t} \tag{4}$$

Algorithm 1 Initialisation du schedule DDPM — `Schedules.py` → `DDPMSchedule.__init__`

Entree : T, $\beta_{\min}$, $\beta_{\max}$

1: $\boldsymbol{\beta} \leftarrow$ torch.linspace($\beta_{\min}$, $\beta_{\max}$, T) ▷ vecteur GPU
2: $\boldsymbol{\alpha} \leftarrow \mathbf{1} - \boldsymbol{\beta}$
3: $\bar{\boldsymbol{\alpha}} \leftarrow$ torch.cumprod($\boldsymbol{\alpha}$) ▷ produit cumule vectorise
4: $\boldsymbol{\sigma} \leftarrow \sqrt{\mathbf{1} - \bar{\boldsymbol{\alpha}}}$

Sortie : $\boldsymbol{\beta}, \boldsymbol{\alpha}, \bar{\boldsymbol{\alpha}}, \boldsymbol{\sigma}$ (tenseurs de taille T)

II.2 Processus forward

Resultat II.2.1— Loi marginale forward (VP-DDPM) [Rapport eq. III.3.1]

$$X_t = \sqrt{\bar{\alpha}_t}\, X_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I_n) \tag{5}$$

soit $X_t | X_0 \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}\, X_0,\ (1 - \bar{\alpha}_t)\, I_n)$.

Algorithm 2 Processus forward — `Forward.py` → `forward_process`

Entree : $x_0 \in \mathbb{R}^{B \times C \times H \times W}$ (batch, torch), $t \in \{0, \dots, T-1\}^B$, $\bar{\boldsymbol{\alpha}}$, $\boldsymbol{\sigma}$

1: $\sqrt{\bar{\alpha}_t} \leftarrow \bar{\boldsymbol{\alpha}}[t]$.view$(B, 1, 1, 1)$ ▷ broadcast pour images
2: $\sigma_t \leftarrow \boldsymbol{\sigma}[t]$.view$(B, 1, 1, 1)$
3: $\varepsilon \leftarrow$ torch.randn_like(x_0)
4: $x_t \leftarrow \sqrt{\bar{\alpha}_t} \cdot x_0 + \sigma_t \cdot \varepsilon$

Sortie : x_t, ε

II.3 Formule de Tweedie

Resultat II.3.1— Estimateur de Tweedie (VP-DDPM) [Rapport eq. 12]

$$\hat{x}_0(x_t) = \mathbb{E}[X_0|X_t = x_t] = \frac{x_t - \sigma_t\, \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \tag{6}$$

Algorithm 3 Estimateur de Tweedie — `Forward.py` → `tweedie_estimate`

Entree : x_t (torch), $\varepsilon_{\text{pred}}$ (torch), t, $\bar{\boldsymbol{\alpha}}$, $\boldsymbol{\sigma}$

$\sqrt{\bar{\alpha}_t} \leftarrow \sqrt{\bar{\boldsymbol{\alpha}}[t]}, \quad \sigma_t \leftarrow \boldsymbol{\sigma}[t]$

$\hat{x}_0 \leftarrow (x_t - \sigma_t \cdot \varepsilon_{\text{pred}}) \,/\, \sqrt{\bar{\alpha}_t}$

Sortie : $\hat{x}_0$

II.4 Objectif d'entrainement

Resultat II.4.1— Loss de Denoising Score Matching [Rapport eq. 15]

$$\mathcal{L}(\theta) = \mathbb{E}_{t,x_0,\varepsilon}\left[\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2\right] \tag{7}$$

ou $t \sim \mathcal{U}\{0, \ldots, T-1\}$, $x_0 \sim p_{\text{data}}$, $\varepsilon \sim \mathcal{N}(0, I)$, et x_t est donne par (5).

Algorithm 4 Boucle d'entrainement — `Training.py` → `train`

Entree : reseau ε_θ, schedule, dataloader, E epoques, lr, λ_{EMA}, γ_{clip}

$\varepsilon_{\text{EMA}} \leftarrow \text{copie}(\varepsilon_\theta)$ ▷ Exponential Moving Average
optimizer $\leftarrow$ Adam(ε_θ, lr)
pour epoch $= 1, \ldots, E$ **faire**
  pour chaque batch x_0 du dataloader **faire** ▷ $x_0 \in [-1, 1]$
    $t \leftarrow \text{randint}(0, T, (B,))$ ▷ echantillonnage uniforme
    $x_t, \varepsilon \leftarrow$ `forward_process`$(x_0, t, \bar{\boldsymbol{\alpha}}, \boldsymbol{\sigma})$ ▷ eq. (5)
    $\varepsilon_{\text{pred}} \leftarrow \varepsilon_\theta(x_t, t)$
    loss $\leftarrow \text{MSE}(\varepsilon_{\text{pred}}, \varepsilon)$ ▷ eq. (7)
    loss.backward()
    clip_grad_norm$(\varepsilon_\theta, \gamma_{\text{clip}})$
    optimizer.step()
    pour $(p_{\text{ema}}, p) \in \text{zip}(\varepsilon_{\text{EMA}}, \varepsilon_\theta)$ **faire** ▷ mise a jour EMA
      $p_{\text{ema}} \leftarrow \lambda_{\text{EMA}}\, p_{\text{ema}} + (1 - \lambda_{\text{EMA}})\, p$
    fin pour
  fin pour
  Sauvegarder checkpoint (etat du modele, EMA, optimizer, epoch)
fin pour

Sortie : ε_θ, ε_{EMA}

Remarque— Subtilite : EMA

A l'inference, on charge les poids EMA (`ema_state_dict`) plutot que les poids du modele, car ils produisent des echantillons de meilleure qualite. C'est fait dans `load_net` de `main.py` via `ckpt.get('ema_state_dict', ckpt['model_state_dict'])`.

III Phase 2 : Degradation de l'image

III.1 Modele d'observation

Resultat III.1.1— Probleme inverse lineaire [Rapport eq. H2]

$$y = A\,x_0 + b, \qquad b \sim \mathcal{N}(0, \sigma_b^2\, I_m) \tag{8}$$

Dans le code actuel : $b = 0$ (pas de bruit de mesure ajoute), A est l'operateur de flou gaussien.

III.2 Construction de l'operateur A

Algorithm 5 Construction de la matrice A — `degrade.py` → `build_A`

Entree : σ_{blur}, taille $H \times W$ $(n = H \cdot W)$

1: $A \leftarrow \mathbf{0}_{n\times n}$ ▷ numpy, CPU
2: **pour** $j = 0, \ldots, n-1$ **faire**
3: $\quad e_j \leftarrow$ vecteur canonique de $\mathbb{R}^n$
4: $\quad A[:, j] \leftarrow \text{blur}(e_j.\text{reshape}(H, W),\ \sigma_{\text{blur}}).\text{ravel}()$
5: **fin pour**

Sortie : $A \in \mathbb{R}^{n\times n}$

Remarque— Subtilite : conditions aux bords

Le flou utilise `mode='wrap'` (conditions periodiques), ce qui rend A une matrice circulante par blocs. Le fichier `operators.py` propose une version FFT (`GaussianBlurOperator`) qui exploite cette structure pour $A\,v$ en $O(n \log n)$ au lieu de $O(n^2)$, mais la construction explicite de A est necessaire pour EMG-VBA (acces a $A^T A$, $(A^T A)_{ii}$).

III.3 Precalculs pour EMG-VBA

Avant la boucle reverse, on precalcule une fois pour toutes (dans `main.py`) :

Algorithm 6 Precalculs — `main.py`

Entree : $A \in \mathbb{R}^{n\times n}$, $y \in \mathbb{R}^n$ (en $[-1, 1]$, vectorise)

1: $A^T A \leftarrow A^T \cdot A$ ▷ numpy, $O(n^2 \cdot m)$, fait une seule fois

2: $A^T y \leftarrow A^T \cdot y_{\text{flat}}$

3: $(A^T A)_{\text{diag}} \leftarrow \text{diag}(A^T A)$ ▷ vecteur des elements diagonaux

IV Phase 3 : Inference conditionnelle (boucle reverse)

IV.1 Architecture globale

Pour chaque pas $t = T-1, T-2, \ldots, 0$:

1. **Tweedie** : estimer $\hat{x}_0$ via le reseau ε_θ (eq. (6))
2. **EMG-VBA** : corriger $\hat{x}_0 \to \mu_{\text{post}} \approx \mathbb{E}[X_0|X_t, Y]$ (eq. (11)–(10))
3. **Reverse step** : echantillonner $x_{t-1}|x_t, \mu_{\text{post}}$ (eq. (40))

IV.2 Score conditionnel et μ_{post}

Resultat IV.2.1— Score conditionnel [Rapport eq. 44]

$$\nabla_{x_t} \log p_t(x_t|y) = \frac{\sqrt{\bar{\alpha}_t}\,\mu_{\text{post}} - x_t}{1 - \bar{\alpha}_t} \tag{9}$$

avec :

$$\Sigma_{\text{post}}^{-1} = \frac{A^T A}{\sigma_b^2} + \frac{I_n}{r_t^2} \tag{10}$$

$$\mu_{\text{post}} = \Sigma_{\text{post}} \left(\frac{A^T y}{\sigma_b^2} + \frac{\hat{x}_0(x_t)}{r_t^2} \right) \tag{11}$$

L'idee cle est de **remplacer** $\hat{x}_0$ par μ_{post} dans le pas reverse, ce qui est **exactement equivalent** a utiliser le score conditionnel total (Resultat X.5.1 du rapport). Les hyperparametres $\tau_r = 1/r_t^2$ et $\tau_b = 1/\sigma_b^2$ sont estimes automatiquement par EMG-VBA.

IV.3 Modele hierarchique bayesien

Resultat IV.3.1— Modele generatif [Rapport eq. M1–M5]

$$x_t|x_0 \sim \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\, I_n\big) \qquad \text{(M1 : forward)} \tag{12}$$
$$x_0|\tau_r \sim \mathcal{N}\big(\hat{x}_0(x_t),\ \tau_r^{-1} I_n\big) \qquad \text{(M2 : a priori ΠGDM)} \tag{13}$$
$$y|x_0, \tau_b \sim \mathcal{N}\big(Ax_0,\ \tau_b^{-1} I_m\big) \qquad \text{(M3 : vraisemblance)} \tag{14}$$
$$\tau_r \sim \text{Gamma}(a_0, b_0) \qquad \text{(M4 : a priori sur } \tau_r\text{)} \tag{15}$$
$$\tau_b \sim \text{Gamma}(c_0, d_0) \qquad \text{(M5 : a priori sur } \tau_b\text{)} \tag{16}$$

IV.4 Approximation variationnelle

Resultat IV.4.1— Hypothese champ moyen [Rapport eq. MF, 60]

On approche l'a posteriori $p_t(w|y)$ par :

$$q(w) = \left(\prod_{i=1}^{n} q_i(x_{0,i})\right) \cdot q(\tau_r) \cdot q(\tau_b) \tag{17}$$

avec $q_i(x_{0,i}) = \mathcal{N}(x_{0,i}|\mu_i, \Sigma_i)$ pour chaque pixel i.

Resultat IV.4.2— Objectif variationnel [Rapport eq. 58–59]

$$\min_q \text{KL}[q\|p_t(\cdot|y)] \iff \max_q\ \mathcal{F}(q) = \mathbb{E}_q\left[\log \frac{p_t(w,y)}{q(w)}\right] \tag{18}$$

IV.5 Log-vraisemblance jointe

Resultat IV.5.1— Log-jointe [Rapport eq. 55]

$$\begin{aligned}\log p_t(w,y) = &-\frac{\|x_t - \sqrt{\bar{\alpha}_t}\, x_0\|^2}{2(1-\bar{\alpha}_t)} \\ &+ \left(\tfrac{m}{2} + c_0 - 1\right)\log \tau_b - \left(d_0 + \tfrac{1}{2}\|y - Ax_0\|^2\right)\tau_b \\ &+ \left(\tfrac{n}{2} + a_0 - 1\right)\log \tau_r - \left(b_0 + \tfrac{1}{2}\|x_0 - \hat{x}_0\|^2\right)\tau_r + \text{cte}\end{aligned}$$

V Phase 3a : Algorithme EMG-VBA (detail)

L'algorithme EMG-VBA est execute K fois *a chaque pas de temps* t de la boucle reverse. Il met a jour les parametres variationnels $(\mu, \Sigma, \tau_r, \tau_b)$ en alternant quatre etapes.

V.1 Etape 1 : Mise a jour de $\langle \tau_b \rangle$ et $\langle \tau_r \rangle$

Resultat V.1.1— Distribution de reference pour τ_b [Rapport eq. 83–85]

$$\tilde{a}_b = \frac{m}{2} + c_0 \tag{19}$$

$$\tilde{b}_b = d_0 + \frac{1}{2}\left(\|y - A\mu\|^2 + (A^T A)^T_{\text{diag}}\Sigma\right) \tag{20}$$

$$\langle \tau_b \rangle = \tilde{a}_b \,/\, \tilde{b}_b \tag{21}$$

Resultat V.1.2— Distribution de reference pour τ_r [Rapport eq. 92–94]

$$\tilde{a}_r = \frac{n}{2} + a_0 \tag{22}$$

$$\tilde{b}_r = b_0 + \frac{1}{2}\left(\|\mu - \hat{x}_0\|^2 + \sum_i \Sigma_i\right) \tag{23}$$

$$\langle \tau_r \rangle = \tilde{a}_r \,/\, \tilde{b}_r \tag{24}$$

Algorithm 7 Mise a jour τ_r, τ_b — `emg_vba_1_dim.py` → `mise_a_jour_taur_taub`

// τ_b (precision du bruit de mesure)
$\tilde{a}_b \leftarrow m/2 + c_0$ ▷ eq. (19)
$\text{res} \leftarrow y - A\mu$ ▷ numpy, vecteur de taille m
$\tilde{b}_b \leftarrow d_0 + \frac{1}{2}\left(\text{res}^T\text{res} + (A^T A)^T_{\text{diag}}\Sigma\right)$ ▷ eq. (20), produits scalaires vectorises
$\langle \tau_b \rangle \leftarrow \tilde{a}_b / \tilde{b}_b$

// τ_r (precision du denoiser)
$\tilde{a}_r \leftarrow n/2 + a_0$ ▷ eq. (22)
$\text{diff} \leftarrow \mu - \hat{x}_0$
$\tilde{b}_r \leftarrow b_0 + \frac{1}{2}\left(\text{diff}^T\text{diff} + \mathbf{1}^T\Sigma\right)$ ▷ eq. (23)
$\langle \tau_r \rangle \leftarrow \tilde{a}_r / \tilde{b}_r$

V.2 Etape 2 : Distribution de reference $q_i^r(x_{0,i})$

Resultat V.2.1— Distribution de reference pour chaque pixel $x_{0,i}$ [Rapport eq. 102–104]

$$(\Sigma_i^r)^{-1} = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} + \langle \tau_b \rangle (A^T A)_{ii} + \langle \tau_r \rangle \tag{25}$$

$$\mu_i^r = \Sigma_i^r \left[\frac{\sqrt{\bar{\alpha}_t}}{1-\bar{\alpha}_t}(x_t)_i + \langle \tau_b \rangle \big((A^T y)_i - (A^T A \mu^k)_i + (A^T A)_{ii} \mu_i^k \big) + \langle \tau_r \rangle (\hat{x}_0)_i \right] \tag{26}$$

Algorithm 8 Distribution de reference — `emg_vba_1_dim.py` → `calculer_distributions_reference_x0`

1: // Toutes les operations sont vectorisees (vecteurs de taille n)
2: $(\Sigma^r)^{-1} \leftarrow \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} + \langle \tau_b \rangle \cdot (A^T A)_{\text{diag}} + \langle \tau_r \rangle$ ▷ eq. (25)
3: $\Sigma^r \leftarrow 1/(\Sigma^r)^{-1}$ ▷ division element par element
4: terme_mesure $\leftarrow A^T y - (A^T A)\mu + (A^T A)_{\text{diag}} \odot \mu$ ▷ eq. (26), astuce de soustraction
5: $\mu^r \leftarrow \Sigma^r \odot \left(\frac{\sqrt{\bar{\alpha}_t}}{1-\bar{\alpha}_t} x_t + \langle \tau_b \rangle \cdot \text{terme_mesure} + \langle \tau_r \rangle \cdot \hat{x}_0 \right)$

Remarque— Subtilite : astuce du terme mesure

Le terme $(A^T y)_i - (A^T A \mu^k)_i + (A^T A)_{ii} \mu_i^k$ s'interprete comme $(A^T y - A^T A \mu_{-i}^k)_i$ (eq. 105 du rapport), c'est-a-dire le residu de vraisemblance en excluant le pixel i courant. En code, on calcule d'abord le produit complet $A^T A \mu$ puis on corrige par l'ajout de $(A^T A)_{ii} \mu_i$, ce qui evite de former μ_{-i} explicitement.

V.3 Etape 3 : Pas sous-optimal

Resultat V.3.1— Mise a jour multiplicative EMG (cas $s_2 = 0$) [Rapport eq. 77]

$$q_i^{k+1}(x_{0,i}) \propto q_i^k(x_{0,i})^{1-s_1} \cdot q_i^{r,k}(x_{0,i})^{s_1} \tag{27}$$

soit pour des gaussiennes :

$$(\Sigma_i^{k+1})^{-1} = \frac{1-s_1}{\Sigma_i^k} + \frac{s_1}{\Sigma_i^{r,k}} \tag{28}$$

$$\mu_i^{k+1} = \Sigma_i^{k+1} \left(\frac{(1-s_1)\mu_i^k}{\Sigma_i^k} + \frac{s_1 \mu_i^{r,k}}{\Sigma_i^{r,k}} \right) \tag{29}$$

Le pas s_1 est determine par un developpement de Taylor d'ordre 2 de l'energie libre.

Resultat V.3.2— Gradient et Hessienne de g^{k+1} (cas 1D : $s_2 = 0$) [Rapport eq. 132–147]

Notations vectorielles :

$$G_{\mu,i} = \frac{\mu_i^r - \mu_i^k}{\Sigma_i^r}, \qquad G_{\Sigma,i} = \frac{1}{2}\left(\frac{1}{\Sigma_i^k} - \frac{1}{\Sigma_i^r} \right) \tag{30}$$

Derivees premieres par rapport a s_1 :

$$\partial_1 \Sigma_i^{k+1} = \Sigma_i^k - \frac{(\Sigma_i^k)^2}{\Sigma_i^r} \tag{31}$$

$$\partial_1 \mu_i^{k+1} = \frac{\Sigma_i^k}{\Sigma_i^r} \left(\mu_i^r - \mu_i^k\right) \tag{32}$$

Gradient scalaire :

$$\nabla g_1 = G_\mu^T(\partial_1 \mu) + G_\Sigma^T(\partial_1 \Sigma) \tag{33}$$

Derivees secondes :

$$\partial_{11} \Sigma_i^{k+1} = \frac{2\,(\partial_1 \Sigma_i^{k+1})^2}{\Sigma_i^k} \tag{34}$$

$$\partial_{11} \mu_i^{k+1} = \frac{2\,(\partial_1 \Sigma_i^{k+1})(\partial_1 \mu_i^{k+1})}{\Sigma_i^k} \tag{35}$$

Hessienne scalaire :

$$H_{11} = G_\mu^T(\partial_{11}\mu) + G_\Sigma^T(\partial_{11}\Sigma) - (\partial_1 \mu)^T M\,(\partial_1 \mu) - \frac{1}{2}(\partial_1 \Sigma)^T D\,(\partial_1 \Sigma) \tag{36}$$

ou :

$$M = \left(\frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} + \langle \tau_r \rangle\right) I_n + \langle \tau_b \rangle A^T A \tag{37}$$

$$D = \mathrm{diag}\left(\frac{1}{(\Sigma_1^k)^2}, \ldots, \frac{1}{(\Sigma_n^k)^2}\right) \tag{38}$$

Pas sous-optimal :

$$s_1^* = -\frac{\nabla g_1}{H_{11}}, \qquad \text{clippe dans } [0,1] \tag{39}$$

Algorithm 9 Pas sous-optimal — `emg_vba_1_dim.py` → `calculer_pas_sousopt`

// Vecteurs G_μ, G_Σ (eq. (30))
$G_\mu \leftarrow (\mu^r - \mu)/\Sigma^r$ ▷ vecteur de taille n
$G_\Sigma \leftarrow \frac{1}{2}\,(1/\Sigma - 1/\Sigma^r)$

// Derivees premieres (eq. (31)–(32))
$\partial_1\Sigma \leftarrow \Sigma - \Sigma^2/\Sigma^r$
$\partial_1\mu \leftarrow (\Sigma/\Sigma^r) \odot (\mu^r - \mu)$

// Gradient (eq. (33))
$\nabla g_1 \leftarrow G_\mu^T \partial_1\mu + G_\Sigma^T \partial_1\Sigma$ ▷ deux produits scalaires

// Derivees secondes (eq. (34)–(35))
$\partial_{11}\Sigma \leftarrow 2\,(\partial_1\Sigma)^2/\Sigma$
$\partial_{11}\mu \leftarrow 2\,(\partial_1\Sigma) \odot (\partial_1\mu)/\Sigma$

// Hessienne : terme M (eq. (36)–(37))
$\text{coeff} \leftarrow \bar{\alpha}_t/(1-\bar{\alpha}_t) + \langle\tau_r\rangle$
$A\,\partial_1\mu \leftarrow A \cdot \partial_1\mu$ ▷ produit matrice-vecteur, $O(mn)$
$\text{terme_M} \leftarrow \text{coeff} \cdot \|\partial_1\mu\|^2 + \langle\tau_b\rangle \cdot \|A\,\partial_1\mu\|^2$

// Hessienne : terme D (eq. (38))
$\text{terme_D} \leftarrow \frac{1}{2}\sum_i (\partial_1\Sigma_i)^2/(\Sigma_i)^2$

$H_{11} \leftarrow G_\mu^T(\partial_{11}\mu) + G_\Sigma^T(\partial_{11}\Sigma) - \text{terme_M} - \text{terme_D}$
$s_1 \leftarrow \text{clip}(-\nabla g_1/H_{11},\ 0,\ 1)$ ▷ eq. (39)

Sortie : s_1

V.4 Etape 4 : Mise a jour de $q(x_0)$

Algorithm 10 Mise a jour de μ et Σ — `emg_vba_1_dim.py` → `mise_a_jour_x0`

Entree : s_1, μ^k, Σ^k, μ^r, Σ^r (tous vecteurs numpy de taille n)

$\mu^{k-1} \leftarrow \mu^k.\text{copy}()$ ▷ sauvegarde pour la memoire (utilise quand $s_2 \neq 0$)
$\Sigma^{k-1} \leftarrow \Sigma^k.\text{copy}()$

$(\Sigma^{k+1})^{-1} \leftarrow (1-s_1)/\Sigma^k + s_1/\Sigma^r$ ▷ eq. (28)
$\Sigma^{k+1} \leftarrow 1/(\Sigma^{k+1})^{-1}$
$\mu^{k+1} \leftarrow \Sigma^{k+1} \odot \big((1-s_1)\mu^k/\Sigma^k + s_1\mu^r/\Sigma^r\big)$ ▷ eq. (29)

$\mu \leftarrow \mu^{k+1}, \quad \Sigma \leftarrow \Sigma^{k+1}$

V.5 Energie libre negative (diagnostic)

Resultat V.5.1— Energie libre negative [Rapport eq. 59, 113–114]

$$\begin{aligned}\mathcal{F}(q) = &\underbrace{-\frac{1}{2(1-\bar{\alpha}_t)}\left(\|x_t - \sqrt{\bar{\alpha}_t}\mu\|^2 + \bar{\alpha}_t \sum_i \Sigma_i\right)}_{\text{forward}} \\ &+ \underbrace{\left(\tfrac{m}{2} + c_0 - 1\right)\langle \log \tau_b \rangle - \left(d_0 + \tfrac{1}{2}\left(\|y - A\mu\|^2 + (A^T A)^T_{\text{diag}}\Sigma\right)\right)\langle \tau_b \rangle}_{\text{mesure}} \\ &+ \underbrace{\left(\tfrac{n}{2} + a_0 - 1\right)\langle \log \tau_r \rangle - \left(b_0 + \tfrac{1}{2}\left(\|\mu - \hat{x}_0\|^2 + \|\Sigma\|_1\right)\right)\langle \tau_r \rangle}_{\text{a priori}} \\ &+ \underbrace{\frac{1}{2}\sum_i \log \Sigma_i + \text{cte}}_{\text{entropie}}\end{aligned}$$

ou $\langle \log \tau_r \rangle = \psi(\tilde{a}_r) - \log(\tilde{b}_r)$ et $\langle \log \tau_b \rangle = \psi(\tilde{a}_b) - \log(\tilde{b}_b)$ avec ψ la fonction digamma.
Propriete fondamentale : $\mathcal{F}(q)$ doit **croitre** a chaque iteration k (Resultat XII.2.1).

V.6 Synthese : boucle EMG-VBA interne

Algorithm 11 EMG-VBA complet a t fixe — `emg_vba_1_dim.py` → `executer`

Entree : A, y, $\bar{\alpha}_t$, x_t, $\hat{x}_0$, (a_0, b_0, c_0, d_0), K iterations, μ_{init}, Σ_{init}

```
μ ← μ_init,   Σ ← Σ_init
pour k = 0, ..., K − 1 faire
    Etape 1 : Mise a jour ⟨τ_b⟩, ⟨τ_r⟩                ▷ eq. (19)–(24)
    Etape 2 : Calculer μ^r, Σ^r                        ▷ eq. (25)–(26)
    Etape 3 : Calculer s_1                             ▷ eq. (39)
    Etape 4 : Mettre a jour μ, Σ                       ▷ eq. (28)–(29)
    Calculer F(q)                                      ▷ eq. (V.5.1), diagnostic
fin pour
```

Sortie : μ ($= \mu_{\text{post}}$), Σ, $\langle \tau_r \rangle$, $\langle \tau_b \rangle$, historique

V.7 Initialisation de EMG-VBA

Remarque— Subtilite critique : initialisation

Premier appel ($t = T-1$, pas de warm start) :

- $\mu_{\text{init}} = \hat{x}_0$ (centre sur l'estimateur de Tweedie)
- $\Sigma_{\text{init}} = r_t^2 \cdot \mathbf{1}_n$ avec $r_t^2 = (1 - \bar{\alpha}_t)/\bar{\alpha}_t$ **[Rapport eq. VII.6.1]**

Appels suivants ($t < T-1$, warm start active) :

— $\mu_{\text{init}} = \mu$ du pas precedent (etat retourne par `state`)
— $\Sigma_{\text{init}} = \Sigma$ du pas precedent

Le warm start permet de repartir d'un bon point initial au lieu de reinitialiser depuis Tweedie a chaque pas, ce qui accelere la convergence de EMG-VBA.

VI Phase 3b : Pas reverse DDPM conditionnel

Resultat VI.0.1— Pas reverse DDPM conditionnel [Rapport eq. 45]

$$x_{t-1} = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mu_{\text{post}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,x_t + \sqrt{\tilde{\beta}_t}\,z \tag{40}$$

avec $z \sim \mathcal{N}(0, I_n)$ et $\tilde{\beta}_t = \frac{(1-\bar{\alpha}_{t-1})\beta_t}{1-\bar{\alpha}_t}$.

Algorithm 12 Pas reverse DDPM — `Reverse.py` → `reverse_step_ddpm`

Entree : x_t (torch), μ_{post} (torch), t, $\bar{\boldsymbol{\alpha}}$, $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$

1: $\bar{\alpha}_t \leftarrow \bar{\boldsymbol{\alpha}}[t]$, $\quad \bar{\alpha}_{t-1} \leftarrow \bar{\boldsymbol{\alpha}}[t-1]$, $\quad \beta_t \leftarrow \boldsymbol{\beta}[t]$, $\quad \alpha_t \leftarrow \boldsymbol{\alpha}[t]$
2: $\text{coef}_{x_0} \leftarrow \sqrt{\bar{\alpha}_{t-1}} \cdot \beta_t \,/\, (1-\bar{\alpha}_t)$
3: $\text{coef}_{x_t} \leftarrow \sqrt{\alpha_t} \cdot (1-\bar{\alpha}_{t-1}) \,/\, (1-\bar{\alpha}_t)$
4: $\mu \leftarrow \text{coef}_{x_0} \cdot \mu_{\text{post}} + \text{coef}_{x_t} \cdot x_t$
5: $\tilde{\beta}_t \leftarrow (1-\bar{\alpha}_{t-1}) \cdot \beta_t \,/\, (1-\bar{\alpha}_t)$
6: $z \leftarrow \text{torch.randn_like}(x_t)$

Sortie : $\mu + \sqrt{\tilde{\beta}_t} \cdot z$

VII Phase 3c : Boucle reverse complete

Algorithm 13 Echantillonnage conditionnel complet — `Reverse.py` → `sample_conditional`

Entree : reseau ε_θ (torch, GPU), schedule, y (numpy, CPU), A (numpy), (C, H, W), N_s samples

Entree : parametres EMG : K, t_{skip}, (a_0, b_0, c_0, d_0), precalculs $A^T A$, $A^T y$, $(A^T A)_{\text{diag}}$

$x_T \leftarrow \text{torch.randn}(N_s, C, H, W)$ ▷ $x_T \sim \mathcal{N}(0, I)$, GPU

$\text{warm_starts} \leftarrow [\text{None}] \times N_s$ ▷ un etat par sample

pour $t = T-1, T-2, \ldots, 0$ **faire** ▷ boucle reverse

  // – Etape 1 : Tweedie (GPU, torch) –

  $\varepsilon_{\text{pred}} \leftarrow \varepsilon_\theta(x_t, t)$ ▷ passe forward du UNet

  $\hat{x}_0 \leftarrow$ `tweedie_estimate`$(x_t, \varepsilon_{\text{pred}}, t)$ ▷ eq. (6)

  // – Etape 2 : EMG-VBA (CPU, numpy) –

  si $t \geq t_{\text{skip}}$ **alors** ▷ correction activee

    pour $b = 0, \ldots, N_s - 1$ **faire**

      $x_t^{(\text{np})} \leftarrow x_t[b].\text{cpu}().\text{numpy}().\text{ravel}()$ ▷ torch → numpy

      $\hat{x}_0^{(\text{np})} \leftarrow \hat{x}_0[b].\text{cpu}().\text{numpy}().\text{ravel}()$

      $\mu_{\text{post}}^{(b)}, \text{state} \leftarrow$ `emg_vba_correction`(

        $x_t^{(\text{np})}, \hat{x}_0^{(\text{np})}, \bar{\alpha}_t, y, A, K, \ldots, \text{warm_start}=\text{warm_starts}[b]$)

      $\text{warm_starts}[b] \leftarrow \text{state}$ ▷ warm start pour le pas $t-1$

      $\mu_{\text{post}}^{(b)} \leftarrow \text{torch}(\mu_{\text{post}}^{(b)}.\text{reshape}(C, H, W))$ ▷ numpy → torch

    fin pour

    $\mu_{\text{post}} \leftarrow \text{torch.stack}([\mu_{\text{post}}^{(0)}, \ldots]).\text{to}(\text{GPU})$

  sinon

    $\mu_{\text{post}} \leftarrow \hat{x}_0$ ▷ pas de correction apres t_{skip}

  fin si

  // – Etape 3 : Pas reverse (GPU, torch) –

  si $t = 0$ **alors**

    $x_t \leftarrow \mu_{\text{post}}$ ▷ pas de bruit au dernier pas

  sinon

    $x_t \leftarrow$ `reverse_step_ddpm`$(x_t, \mu_{\text{post}}, t)$ ▷ eq. (40)

  fin si

fin pour

// – Reconversion $[-1, 1] \to [0, 1]$ –

$x_{\text{rec}} \leftarrow (\text{clamp}(x_t, -1, 1) + 1)/2$

Sortie : x_{rec}, diagnostics

Remarque— Subtilites d'implementation

1. **Transferts torch $\leftrightarrow$ numpy.** Le reseau ε_θ fonctionne sur GPU (torch), tandis que EMG-VBA fonctionne sur CPU (numpy) pour l'algebre lineaire dense ($A^T A$, produits matrice-vecteur). A chaque pas t, il y a donc un aller-retour GPU→CPU→GPU.
2. **Warm start inter-pas.** L'etat $(\mu, \Sigma, \tau_r, \tau_b)$ est propage du pas t au pas $t-1$ via le dictionnaire `state`. Cela evite de repartir de zero et permet a EMG-VBA de converger en peu d'iterations ($K \approx 10$).

VIII Phase 4 : Orchestration

Algorithm 14 Pipeline complete — `main.py` → `main`

```
// – 1. Degradation –
x_0 ← load_and_resize(image, H)                    ▷ [0, 1]
y ← blur(x_0, σ_blur)                              ▷ [0, 1]
x_0 ← 2x_0 − 1,   y ← 2y − 1                       ▷ → [−1, 1]

// – 2. Precalculs (une seule fois) –
A ← build_A(σ_blur, H)                             ▷ ℝ^{n×n}
A^T A, A^T y, (A^T A)_diag ← precalculs            ▷ §III.3

// – 3. Chargement modele –
ε_θ ← load_net(ckpt_dir)                           ▷ poids EMA
schedule ← DDPMSchedule(T, β_min, β_max)

// – 4. Reconstruction –
x_rec ← sample_conditional(ε_θ, schedule, y, A, ...)

// – 5. Sauvegarde et affichage –
Sauvegarder x_rec en PNG et NPY
Afficher triplet (originale, floutee, reconstruite)
Afficher courbes F(q) vs k pour les pas t monitores
```

IX Recapitulatif des equations

Eq.	Nom	Rapport	Code
(1)–(4)	Schedule DDPM	Def. III.3.1	`Schedules.py`
(5)	Forward process	Res. III.3.1	`Forward.py` :`forward_process`
(6)	Tweedie	Eq. (12)	`Forward.py` :`tweedie_estimate`
(7)	Loss DSM	Eq. (15)	`Training.py` :`train`
(8)	Modele d'observation	Hyp. (H2)	`degrade.py`
(9)	Score conditionnel	Res. X.4.1 (44)	implicite via μ_{post}
(11)	μ_{post}	Res. X.3.1 (41)	`emg_vba_1_dim.py`
(12)–(16)	Modele hierarchique	Hyp. XI.2.1	`emg_vba_1_dim.py`
(17)	Champ moyen	Hyp. XII.3.1	`emg_vba_1_dim.py`
(18)	ELBO	Res. XII.2.1 (58)	`calculer_energie_libre_negative`
(IV.5.1)	Log-jointe	Res. XI.4.1 (55)	`calculer_energie_libre_negative`
(19)–(21)	Ref. τ_b	Prop. XIII.8.1 (83–85)	`mise_a_jour_taur_taub`
(22)–(24)	Ref. τ_r	Prop. XIII.8.2 (92–94)	`mise_a_jour_taur_taub`
(25)–(26)	Ref. $x_{0,i}$	Prop. XIII.8.3 (102–104)	`calculer_distributions_reference_x0`
(28)–(29)	Maj. $q(x_0)$	Prop. XIII.8.4 (115–116)	`mise_a_jour_x0`
(30)–(39)	Pas sous-optimal	Res. XIII.9.1 (139–147)	`calculer_pas_sousopt`
(V.5.1)	Energie libre	Eq. (59), (113–114)	`calculer_energie_libre_negative`
(40)	Reverse DDPM	Res. X.5.1 (45)	`reverse_step_ddpm`

X Hyperparametres

Parametre	Valeur	Role	Fichier
T	1000	Nombre de pas de diffusion	`main.py`
$\beta_{\min}, \beta_{\max}$	$10^{-4}, 0.02$	Bornes du schedule lineaire	`main.py`
σ_{blur}	1.5	Ecart-type du flou gaussien	`main.py`
Taille image	64×64	Resolution de travail	`main.py`
K (EMG)	10	Iterations EMG-VBA par pas t	`main.py`
t_{skip}	0	Desactivation EMG apres ce pas	`main.py`
a_0, b_0, c_0, d_0	10^{-3}	A prioris vagues (Gamma)	`main.py`
λ_{EMA}	0.9999	Decay EMA	`Training.py`
γ_{clip}	1.0	Gradient clipping	`Training.py`
lr	2×10^{-4}	Learning rate	`Training.py`